

NEURAL SINDY: SPARSE SYSTEM IDENTIFICATION WITH GROKKED MLP LIBRARIES

Aniruddha Pandey

Department of Computer Science
Stevens Institute of Technology

May 5, 2026

OUTLINE

Part I: Motivation and Background

1 Motivation 3

2 Background: SINDy 4

3 Core Idea 6

Part II: Method

1 Pipeline 9

2 Phases 1 & 2 13

Part III: Experiments and Results

1 Experimental Setup 16

2 Path A: STLSQ 17

3 Path B: Gumbel-Softmax Routing 19

4 Final Results 34

5 Phase 4 and Ablations 35

Part IV: Wrap-up

1 Takeaways 38

2 Limitations and Future Work 39

3 Future Work: Differentiable Trees 40

Part I

MOTIVATION AND BACKGROUND

THE PROBLEM

- ▶ **Goal:** discover the governing equations of a physical system from noisy measurements.
- ▶ **Running toy example:** a damped spring,

$$\ddot{x} = -kx - c\dot{x}.$$

- ▶ You measure position $x(t)$ and velocity $v(t)$ but **do not know** the equation.
- ▶ Can the data tell you it is a spring?

WHAT SINDY DOES

Frame equation discovery as sparse regression:

$$\dot{\mathbf{X}} = \Theta(\mathbf{X}) \Xi$$

- ▶ Θ – a **library** of candidate functions
- ▶ Ξ – a **sparse** coefficient vector

Toy library: $[1, x, v, x^2, xv, \sin(x)]$

Hoped-for outcome: $\Xi = [0, -2.0, -0.3, 0, 0, 0] \Rightarrow \dot{v} = -2x - 0.3v$

SINDY'S THREE PAIN POINTS

1. **Library must be specified up front.** If `sin` is missing but the truth needs `sin`, SINDy returns a confidently wrong fit.
2. **Combinatorial explosion.** Higher-order or composed functions blow up library size.
3. **Noise sensitivity.** Rigid analytical primitives do not absorb sensor noise gracefully.

This work attacks all three by **replacing the library with neural networks** .

THE CORE IDEA

Replace each library entry with a small MLP trained to “grok” a math operation:

$$\sin(x) \rightarrow \sin_mlp(x), \quad x + y \rightarrow \text{add_mlp}(x, y), \dots$$

Why this could help:

- ▶ MLPs are [differentiable end-to-end](#) – enables gradient-based selection.
- ▶ Smooth approximations may [absorb noise better](#) .
- ▶ Symbolic interpretability recovered later via [PySR distillation](#) .

PRIMER: GROKING

- ▶ **Grokking** : validation loss drops sharply long after training loss has plateaued.
- ▶ Tiny MLPs (128 hidden, Tanh) trained with AdamW + heavy weight decay until val MSE $\approx 10^{-7}$.
- ▶ Used **descriptively** – not claiming the same mechanism as the original modular-arithmetic paper.

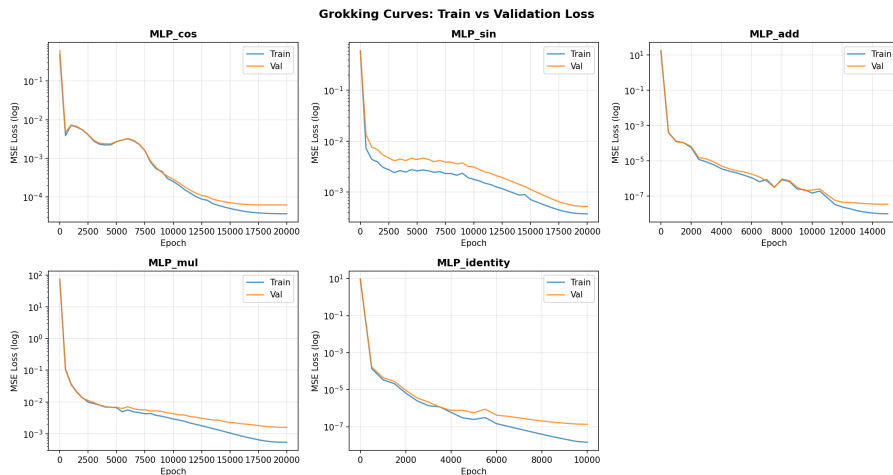


Figure. Training vs. validation loss illustrating grokking.

Part II

METHOD

PIPELINE OVERVIEW

1. **Phase 1.** Grok the MLPs (one per math operation).
2. **Phase 2.** Build Θ by evaluating MLPs on data.
3. **Phase 3.** Discover sparse coefficients.
 - **Path A:** STLSQ (classical).
 - **Path B:** Gumbel-Softmax routing (differentiable).
4. **Phase 4.** Symbolic distillation via PySR.

PRIMER: LEAST SQUARES

For data (x_t, v_t, a_t) , find ξ minimizing total squared error:

$$\min_{\xi} \sum_t (a_t - [\xi_1 + \xi_2 x_t + \xi_3 v_t + \xi_4 x_t^2])^2$$

Closed form:

$$\xi = (\Theta^T \Theta)^{-1} \Theta^T a.$$

PRIMER: SPARSE REGRESSION

Plain least squares fits **all** coefficients (most slightly nonzero from noise).

Sparse regression prefers solutions where most coefficients are exactly zero – because real laws are short.

STLSQ (Sequentially Thresholded Least Squares):

1. Run least squares.
2. Zero out any $|\xi_i| < \text{threshold}$.
3. Refit on survivors.
4. Repeat.

STLSQ ON THE SPRING (TOY)

Truth: $\dot{v} = -2x - 0.3v$. Library: $[1, x, v, x^2]$. Threshold = 0.1.

Step	Coefficients
Iter 1 (LSQ)	$[0.05, -1.95, -0.28, 0.03]$
Threshold	$[0, -1.95, -0.28, 0]$
Iter 2 (refit)	$[0, -2.00, -0.30, 0]$

Recovered: $\dot{v} = -2.00x - 0.30v$. **Correct.**

PHASE 1: GROKING THE LIBRARY

Five small MLPs, one per operation:

$\text{identity}(x)$, $\cos(x)$, $\sin(x)$, $\text{add}(x, y)$, $\text{mul}(x, y)$

Each trained on random samples until validation MSE $\sim 10^{-7}$.

These **frozen** MLPs become the columns of Θ .

PHASE 2: NEURAL LIBRARY CONSTRUCTION

Standard SINDy library:

$$\Theta(X) = [1, x, v, \sin(x), \dots]$$

Neural SINDy library:

$$\Theta(X) = [\text{id_mlp}(x), \text{id_mlp}(v), \text{sin_mlp}(x), \text{cos_mlp}(x), \\ \text{add_mlp}(x, v), \text{mul_mlp}(x, v), \dots]$$

Eight columns total in the paper's experiments.

Part III

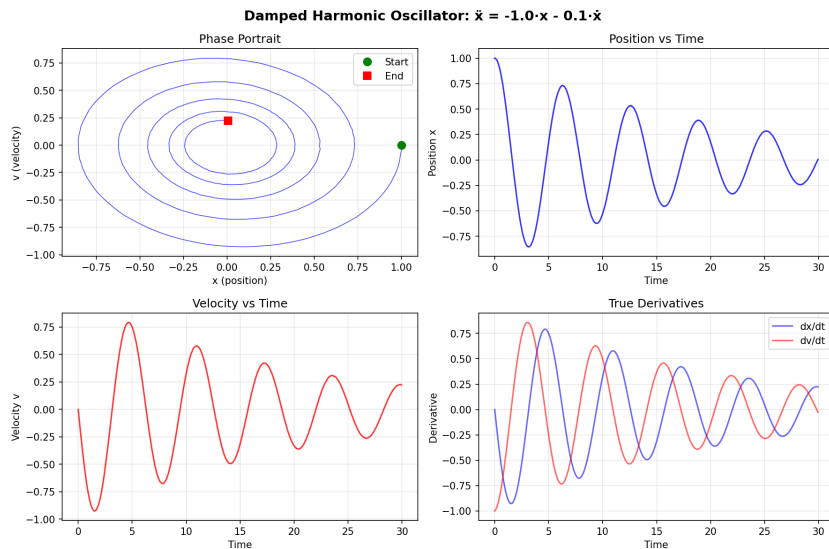
EXPERIMENTS AND RESULTS

EXPERIMENTAL SETUP

System: damped harmonic oscillator, $\ddot{x} = -kx - c\dot{x}$, $k = 1.0$, $c = 0.1$.

Data: 600 time points, $\Delta t = 0.05$, Gaussian noise $\sigma = 0.001$.

Initial conditions: $x(0) = 1.0$, $\dot{x}(0) = 0.0$.



PATH A: STLSQ RESULT

After STLSQ on the 8-term neural library:

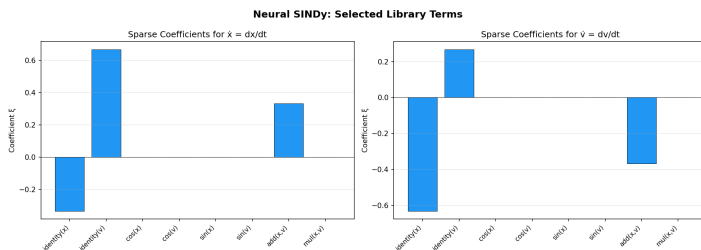
$$\dot{x} = -0.33 \text{id}(x) + 0.67 \text{id}(v) + 0.33 \text{add}(x, v)$$

$$\dot{v} = -0.63 \text{id}(x) + 0.27 \text{id}(v) - 0.37 \text{add}(x, v)$$

Substituting $\text{add}(x, v) \approx x + v$:

$$\dot{x} \approx 0.9999 v, \quad \dot{v} \approx -0.9998 x - 0.0999 v.$$

MSE $\approx 10^{-6}$, but 6 of 16 coefficients nonzero.



PRIMER: COLLINEARITY

Two library columns are **linearly dependent** when one equals a combination of others:

$$\text{add}(x, v) \equiv \text{id}(x) + \text{id}(v).$$

Θ becomes **rank-deficient** $\Rightarrow$ infinitely many equally-good solutions $\Rightarrow$ STLSQ splits coefficients arbitrarily across redundant columns.

Output is correct; **individual coefficients lose interpretability**.

WHY PATH B (ROUTING)?

STLSQ has no notion of “redundant terms.” We need a selection mechanism that:

- ▶ Picks a **discrete** subset of basis functions.
- ▶ Stays **differentiable** for gradient training.
- ▶ Can be biased toward **simpler** terms.

Enter **Gumbel-Softmax routing** .

PRIMER: GUMBEL-SOFTMAX (1/2)

A trick to make “pick one discrete option” differentiable.

Recipe:

1. Sample Gumbel noise $g_i \sim \text{Gumbel}(0, 1)$ per option.
2. Add to logits: $\ell_i + g_i$.
3. Divide by temperature τ .
4. Apply softmax.

$$p_i = \frac{\exp((\ell_i + g_i)/\tau)}{\sum_j \exp((\ell_j + g_j)/\tau)}$$

PRIMER: GUMBEL-SOFTMAX (2/2)

Logits $\ell = [2.0, 0.5, -1.0]$ for [id, sin, cos].

High $\tau = 5.0$ (**early**): softmax $\approx [0.42, 0.37, 0.21]$ – soft mixture, all gradients flow.

Low $\tau = 0.05$ (**late**): softmax $\approx [1.000, 0.000, 0.000]$ – effectively one-hot.

Schedule: τ anneals $5.0 \rightarrow 0.05$ over 2,500/4,000 epochs.

PRIMER: STRAIGHT-THROUGH ESTIMATOR

Forward pass: replace softmax with **hard argmax** $\Rightarrow$ discrete behavior.

Backward pass: pretend you used soft softmax $\Rightarrow$ **gradients flow** .

Without it, argmax has zero gradient and the router cannot learn.

WHY THE GUMBEL NOISE MATTERS (TOY)

Two terms tied at small displacement: $\sin(x) \approx x$ for $|x| < 1$.

Without noise: identical softmax every pass $\Rightarrow$ router locks onto whichever started slightly higher. No exploration.

With noise: different sample-by-sample picks $\Rightarrow$ both terms get tested $\Rightarrow$ complexity prior breaks the tie.

The noise is what makes Gumbel-Softmax actually **sample** rather than smooth.

FOUR ROUTER VARIANTS

R1, R2, R3, R4 – each fixing a flaw in the previous one.

	Routing	k	Prior	Result
R1	state-dependent	1	none	misses damping
R2	state-independent	1	none	sin /id collapse
R3	state-independent	2	$\alpha = 1.0$	correct
R4	state-independent	2	$\alpha = 1.5 + \text{cond. ent.}$	correct

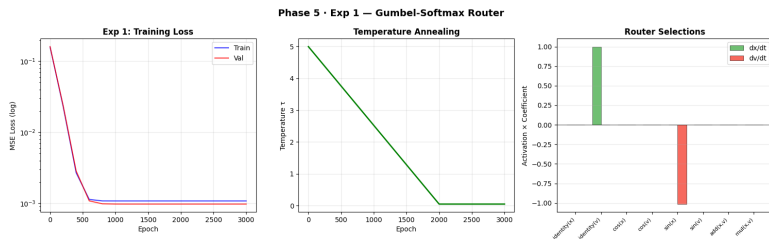
R1: STATE-DEPENDENT ROUTER

Logits depend on the state via a small MLP:

$$\ell(x, v) = \text{MLP}_{\text{router}}([x, v]).$$

Problems: $\sim 10,000$ params; routing varies per sample (**violates SINDy global uniformity**); $k = 1$ allows only one term per equation.

Recovers: $\dot{x} = +1.0 \text{id}(v)$, $\dot{v} = -1.01 \sin(x)$ – damping missing.



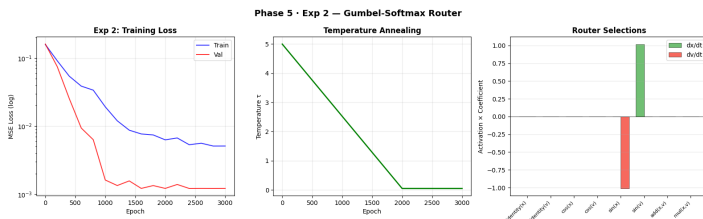
R2: STATE-INDEPENDENT ROUTER

One learnable logit vector per derivative, broadcast across all samples. **Only 32 params.** Restores global uniformity.

But: still $k = 1$; no complexity prior $\Rightarrow$ **collapses to $\sin(x)$ instead of x .**

$$\dot{x} = +1.03 \sin(v), \quad \dot{v} = -1.01 \sin(x).$$

Symbolically misleading.



THE sin /identity COLLINEARITY (TOY)

For $|x| < 1$:

x	$\sin(x)$	error
1.000	0.841	0.159
0.350	0.343	0.007
0.180	0.179	0.001

Columns are numerically interchangeable $\Rightarrow$ no data-driven way to prefer one. R2 picks whichever started higher.

We need an [explicit Occam's-razor bias](#) .

R3 FIX (A): TOP- k ROUTING

Pick $k = 2$ terms per derivative via **iterative masked sampling without replacement** :

1. Sample slot 1 $\Rightarrow$ say $\sin(x)$.
2. Mask it: $\ell_{\sin(x)} = -\infty$.
3. Sample slot 2 from remaining $\Rightarrow$ say $\text{id}(v)$.

Now $\dot{v}$ can have **two terms** , enabling damping.

R3 FIX (B): COMPLEXITY PRIOR

Add a fixed bias to logits based on simplicity:

Term	Bias
identity	$+\alpha$
sin, cos	0
add, mul	$-\alpha$

Toy: raw logits $[1.0, 1.1, 0.5, 0.2]$ for $[\text{id}(x), \sin(x), \text{id}(v), \text{add}]$ become $[2.0, 1.1, 1.5, -0.8]$ with $\alpha = 1.0$.
Identity wins the tie.

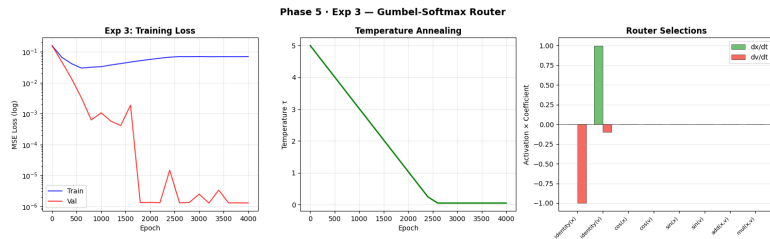
Note: the prior is [manual](#) . Future work: automate via node count or MDL.

R3 RESULT

$$\dot{x} = +0.9999 \text{id}(v)$$
$$\dot{v} = -0.9997 \text{id}(x) - 0.0998 \text{id}(v)$$

Correct to four decimal places. Only 48 params.

But: stochastic Gumbel sampling at val time produced **spikes up to 1000×** the surrounding floor.



PRIMER: CONDITIONAL SLOT ENTROPY (R4)

Original (R3) entropy regularizer: penalized the entropy of the **base** logits – before masking. After slot 1 committed, slot 2's conditional distribution stayed flat.

R4 fix: penalize **each slot's conditional distribution as the mask advances** – mirroring the forward pass.

This forces **every** slot to commit independently. Damping coefficient now lands cleanly.

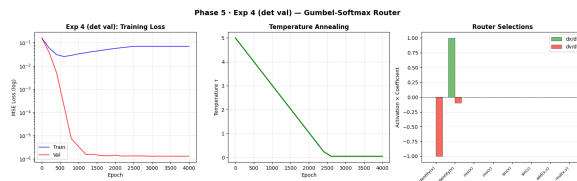
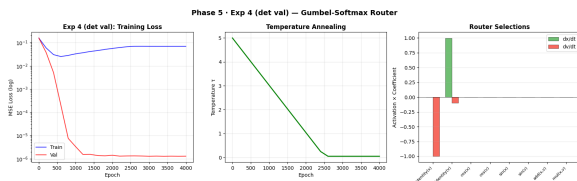
Substantive contribution of the routing pipeline.

PRIMER: DETERMINISTIC VALIDATION

Standard Gumbel-Softmax draws fresh noise every forward pass – including at validation. Single unlucky draws caused $100\times$ – $1000\times$ val-MSE spikes.

Fix: add a hard-argmax evaluation path, used **only at val time**.

- ▶ Training: stays stochastic (optimization unchanged).
- ▶ Validation: deterministic, reproducible.



WHY TRAINING GOES UP WHILE VALIDATION GOES DOWN

Training loss = reconstruction MSE + entropy + complexity penalties.

Validation loss = reconstruction MSE only (deterministic argmax).

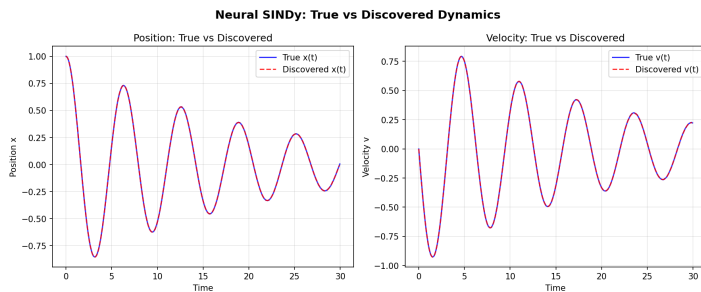
The penalty weight $\eta(\tau) = \eta_{\max}(1 - \tau/\tau_{\text{start}})$ **grows** as τ anneals.

Epoch	Recon	η	Penalties	Train	Val
600	0.02	0.05	0.0015	0.0215	0.02
2000	10^{-4}	0.4	0.035	0.035	10^{-4}
4000	10^{-6}	1.0	0.055	0.055	10^{-6}

Not a generalization gap – two different objectives plotted on the same axes.

FINAL RESULTS TABLE

Variant	Params	Val MSE	$\dot{x}$	$\dot{v}$
R1 (state-dep, $k = 1$)	9,760	9.8×10^{-4}	$+1.0000 \, v$	$-1.0138 \sin(x)$
R2 (state-indep, $k = 1$)	32	1.2×10^{-3}	$+1.0265 \sin(v)$	$-1.0138 \sin(x)$
R3 (top-2, $\alpha=1.0$, det. val)	48	1.0×10^{-6}	$+0.9999 \, v$	$-0.9998 \, x - 0.0996 \, v$
R4 (top-2, $\alpha=1.5$, cond. ent., det. val)	48	1.0×10^{-6}	$+0.9998 \, v$	$-0.9998 \, x - 0.0997 \, v$
Truth	—	—	$+1.0 \, v$	$-1.0 \, x - 0.1 \, v$



PHASE 4: SYMBOLIC DISTILLATION

After routing picks a winning MLP, distill it back to symbols using [PySR](#) (genetic programming over expression trees).

Toy: probe `sin_mlp` on a grid $\rightarrow$ table of (x, y) pairs $\rightarrow$ PySR searches:

- ▶ x – bad fit
- ▶ $x - x^3/6$ – close
- ▶ $\sin(x)$ – near-perfect, very short ✓

Status: described in methodology, **not yet integrated end-to-end**. Listed as future work.

ABLATION: $\alpha = 1.0$ vs. $\alpha = 1.5$

Once **conditional slot entropy** is in place, the two settings produce statistically indistinguishable equations:

- ▶ R3 ($\alpha = 1.0$): $\dot{v} = -0.9998 x - 0.0996 v$
- ▶ R4 ($\alpha = 1.5$): $\dot{v} = -0.9998 x - 0.0997 v$

Conditional slot entropy is the substantive contribution – the prior strength is cosmetic at this precision.

Part IV

WRAP-UP

TAKEAWAYS

1. Grokked MLPs **can serve as a SINDy library** and recover true dynamics to $\sim 10^{-6}$ MSE.
2. Naive neural libraries surface **two flavors of collinearity** ($\text{add} \equiv \text{id} + \text{id}$; $\text{sin} \approx \text{id}$ at small input) that STLSQ cannot resolve.
3. **Differentiable routing** with Gumbel-Softmax handles selection – but only with top- k , complexity prior, and conditional slot entropy.
4. **Conditional slot entropy** is the substantive methodological contribution.
5. Training-vs-validation inversion is **expected** – different objectives, not a generalization gap.

LIMITATIONS AND FUTURE WORK

- ▶ **Tested on one system.** Damped oscillator only – needs higher-dim systems.
- ▶ **Complexity prior is manual.** Should automate via node count / MDL.
- ▶ **PySR not yet integrated.** Phase 4 is methodology, not yet code.
- ▶ **Noise robustness.** Only $\sigma = 0.001$ tested – needs a sweep.
- ▶ **Grokking basis quality.** All MLPs trained to $\text{MSE} \sim 10^{-7}$; weaker grokking would propagate noise into routing.

THE COMBINATORIAL-EXPLOSION WALL

Current pipeline routes over a **flat** 8-term library.

To discover nested equations like $\sin(x_1 + x_2)$, classical SINDy must pre-compute every depth- D composition.

- ▶ Library size scales as $\mathcal{O}(N^D)$ with N operators and depth D .
- ▶ Even modest $N=10$, $D=4$ blows past 10^4 columns.

This is the wall a flat library hits, including ours.

STACKED GUMBEL-SOFTMAX ROUTING TREES

Replace the flat library with a **multi-layer routing tree** of grokked MLPs.

- ▶ Each layer holds only base operators (e.g., $+$, $\times$, $\sin$, $\cos$).
- ▶ A Gumbel-Softmax router Jang et al., 2017 between layers acts as a differentiable switchboard.
- ▶ Soft probabilistic routing at high τ , snapping to discrete paths as $\tau \rightarrow 0$.

Memory: $\mathcal{O}(N \cdot D)$ instead of $\mathcal{O}(N^D)$ – linear, not combinatorial.

Conceptually adjacent to EQL Martius and Lampert, 2016, VaSST Roy et al., 2026, and Symbolic-KAN Faroughi et al., 2026; **novelty here is grokked MLPs as the per-layer primitives** .

TOY EXAMPLE: DISCOVERING $y = \sin(x_1 + x_2)$

Layer 0 (inputs): x_1, x_2 .

Layer 1 (binary ops): `add_mlp`, `mul_mlp`.

- ▶ Router $W^{(1)}$ chooses inputs for each binary op.
- ▶ Snaps to: $h_1 = \text{add_mlp}(x_1, x_2) \approx x_1 + x_2$.

Layer 2 (unary ops): `sin_mlp`, `cos_mlp`.

- ▶ Router $W^{(2)}$ picks among Layer-0 and Layer-1 outputs.
- ▶ Snaps to: $y = \text{sin_mlp}(h_1) \approx \sin(x_1 + x_2)$. ✓

Only 4 primitives in memory, depth-2 nested expression recovered.

UNIVERSAL GATE VARIANT: GROKKED EML

A single binary operator suffices in principle:

$$\text{eml}(x, y) = \exp(x) - \ln(y),$$

recently shown to generate **every elementary function** via recursion.

- ▶ **Catch:** analytic EML is unstable – $\ln(y)$ singularity at 0, exponential drift.
- ▶ **Our angle:** a **grokked MLP** approximating EML smooths the singularity and bounds the gradients.

Architectural simplification: the routing tree becomes **homogeneous** – routers no longer pick operators, only operands.

Bonus: EML structure admits a deterministic compiler back to algebraic notation – **may obviate PySR for Phase 4.**

PROPOSED ROADMAP

1. **Ablation first.** Standard SINDy library + Gumbel routing vs. grokked-MLP library + Gumbel routing – isolate the contribution of grokked primitives.
2. **Depth-2 tree** on a system with known nested structure (e.g., pendulum: $\ddot{\theta} = -(g/\ell) \sin \theta$).
3. **Grokked EML gate** as a single-primitive replacement for the heterogeneous library.
4. **Deterministic distillation** from EML routing paths as an alternative to PySR.

FUTURE WORK: DIFFERENTIABLE TREES

Thank you.

Questions?

REFERENCES I

-  Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). **Discovering governing equations from data by sparse identification of nonlinear dynamical systems.** *Proceedings of the National Academy of Sciences*, 113(15), 3932–3937.
-  Cranmer, M. (2023). **Interpretable machine learning for science with pysr and symbolicregression.jl.** *arXiv preprint arXiv:2305.01582*.
-  Faroughi, S. A., Mostajeran, F., Arzani, A., & Faroughi, S. (2026). **Symbolic-KAN: Kolmogorov-arnold networks with discrete symbolic structure for interpretable learning.** *arXiv preprint arXiv:2603.23854*.
-  Jang, E., Gu, S., & Poole, B. (2017). **Categorical reparameterization with gumbel-softmax.** *International Conference on Learning Representations (ICLR)*.
-  Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T. Y., & Tegmark, M. (2024). **KAN: Kolmogorov-arnold networks.** *arXiv preprint arXiv:2404.19756*.
-  Martius, G., & Lampert, C. H. (2016). **Extrapolation and learning equations.** *arXiv preprint arXiv:1610.02995*.
-  Odrzywołek, A. (2026). **The EML operator: A universal continuous binary gate for elementary functions.** *arXiv preprint*.
-  Power, A., Burda, Y., Edwards, H., Babuschkin, I., & Misra, V. (2022). **Grokking: Generalization beyond overfitting on small algorithmic datasets.** *arXiv preprint arXiv:2201.02177*.

REFERENCES II



Roy, S., Dey, P., & Mallick, B. K. (2026). **VaSST: Variational inference for symbolic regression using soft symbolic trees.** *arXiv preprint arXiv:2602.23561*.